

Documentation technique – Projet DISCOVer

Documentation technique – Projet DISCOVer.....	1
1. Contexte du projet.....	3
2. Objectifs.....	3
2.1 Objectifs pédagogiques.....	3
2.2 Objectifs fonctionnels.....	3
3. Périmètre fonctionnel.....	3
3.1 Utilisateur non authentifié.....	3
3.2 Utilisateur authentifié.....	4
4. Architecture générale.....	4
5. Choix technologiques et librairies.....	4
5.1 Backend et API.....	5
5.2 Base de données et conteneurisation.....	5
5.3 Authentification et sécurité.....	5
5.4 Analyse audio et Machine Learning.....	6
5.5 Frontend.....	6
5.5.1 Technologies principales.....	6
5.5.2 Librairies Flutter utilisées et rôle.....	6
5.6 Outils de développement et tests.....	7
6. Modélisation des données.....	8
6.1 Schéma de base de données.....	8
6.2 User.....	8
6.3 Song.....	8
6.4 UploadedBy.....	9
6.5 History.....	10
6.6 Analyze.....	10
7. API REST – Description des routes.....	10
7.1 Authentification.....	10
7.2 Utilisateurs.....	10
7.3 Musiques.....	11
7.4 Uploads.....	11
7.5 Analyse.....	12
7.6 Historique.....	12
8. Machine Learning.....	12
8.1 Principe.....	12
8.2 Features utilisées.....	13
8.3 Entraînement.....	13
9. Frontend - Architecture et fonctionnement.....	13
9.1 Architecture.....	13
9.2 Fonction et méthodes globales.....	14
10. Sécurité et gestion des accès.....	14
11. Organisation du projet.....	14
12. Limites et perspectives.....	15
12.1 Limites.....	15

1. Contexte du projet

Le projet DISCOVer est réalisé dans le cadre du Projet Tutoré 3A Informatique (Semestre 6) à l'Université Claude Bernard Lyon 1.

Il vise à concevoir une application backend permettant l'analyse de données musicales et la prédiction d'un score de popularité à partir de caractéristiques audio de morceaux (inspirées de plateformes de streaming musical).

2. Objectifs

2.1 Objectifs pédagogiques

- Manipuler et analyser un jeu de données réel
- Concevoir une base de données relationnelle
- Développer une API REST structurée
- Implémenter un modèle de Machine Learning simple
- Travailler en équipe sur un projet logiciel complet

2.2 Objectifs fonctionnels

- Rechercher et consulter des musiques
- Uploader un fichier audio et en extraire des caractéristiques
- Calculer un score de popularité prédictif
- Gérer des utilisateurs et leur historique

3. Périmètre fonctionnel

3.1 Utilisateur non authentifié

Un utilisateur non connecté peut :

- consulter et rechercher des musiques publiques,
- analyser une musique (recalcul du score ML à la volée),

- uploader un fichier audio pour obtenir une analyse ponctuelle.

Aucune donnée utilisateur n'est persistée (pas d'historique, pas d'upload privé).

3.2 Utilisateur authentifié

Un utilisateur connecté peut en plus :

- consulter son historique de musiques,
- gérer ses musiques uploadées,
- définir la visibilité de ses uploads (public / privé),
- accéder à ses analyses sauvegardées.

4. Architecture générale

L'application backend repose sur une architecture en couches :

- Routes API (Flask Blueprints)
Exposent les endpoints REST.
- Guards
Gèrent l'authentification obligatoire ou optionnelle.
- CRUD
Centralisent les accès à la base de données.
- Services
Contiennent la logique métier (analyse audio, ML).
- Base de données
Stockage persistant des données.

Cette architecture favorise la lisibilité, la maintenabilité et l'évolutivité du projet.

5. Choix technologiques et librairies

Cette section présente les technologies et librairies utilisées dans le projet DISCOVer, ainsi que leur rôle dans la réalisation de l'application, côté backend et frontend.

Les choix technologiques ont été guidés par la volonté d'utiliser des outils modernes, standards et adaptés à un projet full-stack orienté données.

5.1 Backend et API

- Python 3
Langage principal du backend. Il a été choisi pour sa lisibilité, sa rapidité de développement et son écosystème riche pour l'analyse de données et le Machine Learning.
- Flask
Framework web léger utilisé pour développer l'API REST.
Il permet de définir les routes HTTP, de gérer les requêtes et de structurer l'application via des Blueprints.
- Werkzeug
Bibliothèque utilisée pour la gestion sécurisée des mots de passe (hashage et vérification) ainsi que pour certains utilitaires HTTP utilisés par Flask.

5.2 Base de données et conteneurisation

- MySQL
Système de gestion de base de données relationnelle utilisé pour stocker l'ensemble des données du projet : utilisateurs, musiques, analyses, uploads et historique.
Il assure la persistance des données et la cohérence relationnelle.
- Docker
Utilisé pour exécuter la base de données MySQL dans un conteneur.
La conteneurisation permet de garantir un environnement reproductible, indépendant de la machine hôte, et facilite le déploiement et les tests du projet.
- SQLAlchemy
ORM (Object-Relational Mapping) utilisé pour définir les modèles de données et interagir avec la base MySQL sans écrire de requêtes SQL brutes.
- Flask-SQLAlchemy
Extension facilitant l'intégration de SQLAlchemy au sein de l'application Flask.
- Kaggle
Dataset de départ :
<https://www.kaggle.com/datasets/yasserh/song-popularity-dataset>

5.3 Authentification et sécurité

- PyJWT
Bibliothèque utilisée pour la gestion des tokens JWT stockés dans des cookies sécurisés.
Elle permet l'authentification des utilisateurs et la protection des routes sensibles.
- Marshmallow
Utilisée pour la validation et la désérialisation des données échangées avec l'API.
Elle garantit la cohérence et la validité des payloads JSON reçus par le backend.

5.4 Analyse audio et Machine Learning

- librosa
Bibliothèque dédiée à l'analyse audio.
Elle est utilisée pour extraire les caractéristiques audio des fichiers .mp3 (durée, tempo, énergie, etc.).
- NumPy
Utilisée pour la manipulation de tableaux numériques et les calculs mathématiques nécessaires à l'analyse audio et au Machine Learning.
- pandas
Utilisée pour la manipulation des jeux de données, la préparation des features et la gestion des fichiers CSV lors de l'entraînement du modèle.
- scikit-learn
Bibliothèque principale de Machine Learning.
Elle est utilisée pour l'entraînement du modèle de prédiction de popularité, la normalisation des données et l'évaluation des performances.
- joblib
Utilisée pour sauvegarder et charger le modèle de Machine Learning entraîné.

5.5 Frontend

Le frontend est une application Flutter (langage Dart). Elle constitue l'interface utilisateur et communique avec le backend via l'API REST (requêtes HTTP).

5.5.1 Technologies principales

- Flutter (SDK)
Framework utilisé pour développer une application multi-plateforme (Android/iOS/Web/Desktop). Il permet de construire une interface en widgets et d'assurer une expérience utilisateur cohérente sur plusieurs environnements.

5.5.2 Bibliothèques Flutter utilisées et rôle

- http
Utilisée pour effectuer les appels HTTP vers l'API backend (auth, recherche de musiques, analyse, upload, historique).
- shared_preferences
Utilisée pour persister localement des informations simples (préférences UI, paramètres, éventuels éléments de session/cache léger).
- file_picker
Utilisée pour sélectionner un fichier audio (mp3) depuis l'appareil, afin de l'envoyer à la route d'upload du backend.

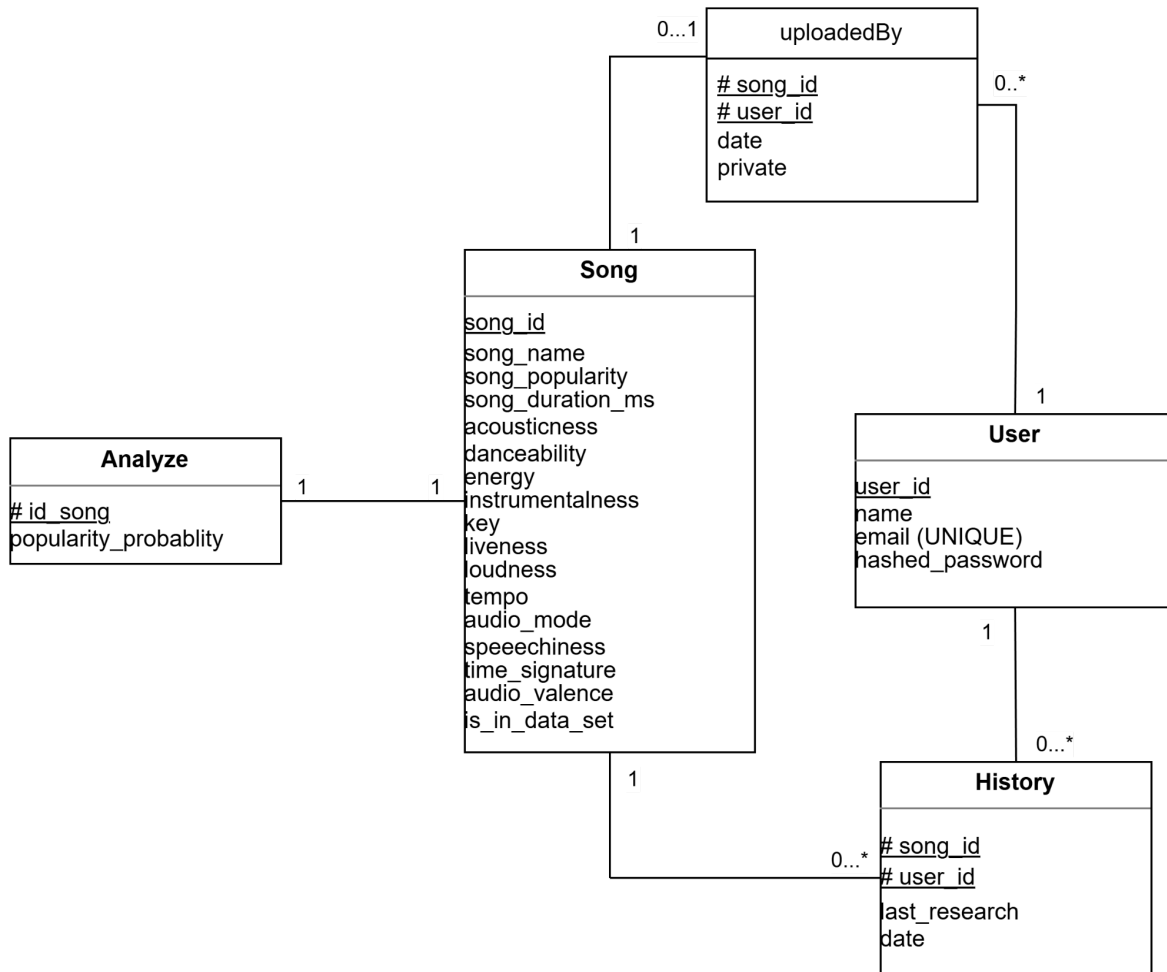
- flutter_svg
Utilisée pour afficher des icônes/images au format SVG dans l'interface.
- google_fonts
Utilisée pour intégrer des polices Google Fonts et harmoniser le style visuel.
- loading_animation_widget
Utilisée pour afficher des animations de chargement (spinners / loaders) pendant les appels réseau.
- vibration
Utilisée pour ajouter un retour haptique (vibration) sur certaines interactions.
- restart_app
Utilisée pour relancer l'application dans certains cas (ex : reset d'état après un logout, ou rafraîchissement global).
- lordicon
Utilisée pour intégrer des animations d'icônes (UI plus dynamique).

5.6 Outils de développement et tests

- Git / GitHub
Utilisés pour le versionnement du code, le travail collaboratif et le suivi de l'évolution du projet.
- PyCharm
Environnement de développement utilisé pour le backend (développement, débogage, organisation du projet).
- Visual Studio Code
Utilisé pour le développement du frontend.
- PowerShell
Utilisé pour tester manuellement les routes de l'API (requêtes HTTP, scénarios de test, upload de fichiers).

6. Modélisation des données

6.1 Schéma de base de données



6.2 User

Représente un compte utilisateur.

- `user_id` (PK)
- `name`
- `email` (unique)
- `hashed_password`

6.3 Song

Représente un morceau musical et ses caractéristiques audio.

- `song_id` (PK)

- song_name
- song_popularity (valeur dataset, nullable)
- song_duration_ms
- acousticness
- danceability
- energy
- instrumentalness
- liveness
- speechiness
- key
- loudness
- tempo
- audio_mode
- time_signature
- audio_valence
- is_in_data_set

6.4 UploadedBy

Lien entre un utilisateur et une musique uploadée.

- user_id (FK)
- song_id (FK)
- private
- date

6.5 History

Historique de consultation des musiques.

- user_id (FK)
- song_id (FK)
- date

L'historique est mis à jour uniquement si l'utilisateur est authentifié.

6.6 Analyze

Résultat d'analyse prédictive associée à une musique.

- id_song (PK / FK)
- predicted_popularity (score 0–100)

7. API REST – Description des routes

7.1 Authentification

- POST /auth/login
Permet à un utilisateur de se connecter à l'application à l'aide de son email et de son mot de passe.
Un cookie de session est créé si l'authentification réussit.
- POST /auth/logout
Déconnecte l'utilisateur en supprimant le cookie de session.
- GET /auth/me
Indique si un utilisateur est actuellement connecté et retourne ses informations de base.

7.2 Utilisateurs

- POST /users
Crée un nouveau compte utilisateur.
- GET /users/me
Retourne les informations du compte de l'utilisateur connecté.
- PATCH /users/me
Permet à l'utilisateur connecté de modifier ses informations personnelles (nom,

email, mot de passe).

- GET /users
Liste les utilisateurs existants (route utilisée principalement pour le développement et les tests).

7.3 Musiques

- GET /songs
Retourne la liste des musiques accessibles :
 - musiques du dataset et musiques publiques pour un utilisateur non connecté,
 - musiques publiques + privées de l'utilisateur s'il est connecté.
- GET /songs/{song_id}
Retourne le détail d'une musique donnée.
Si l'utilisateur est connecté, la consultation est ajoutée à son historique.
- GET /songs/me
Retourne uniquement les musiques uploadées par l'utilisateur connecté.
- POST /songs
Crée une musique à partir de données fournies en JSON (principalement utilisé pour l'initialisation ou les tests).
- DELETE /songs/{song_id}
Supprime une musique uploadée par l'utilisateur connecté.

7.4 Uploads

- POST /uploads/file
Permet d'uploader un fichier audio (.mp3).
Le fichier est analysé, les caractéristiques audio sont extraites et un score de popularité est calculé.
 - utilisateur non connecté : analyse ponctuelle sans historique,
 - utilisateur connecté : musique liée au compte et visible dans ses uploads.
- POST /uploads
Crée un lien entre un utilisateur et une musique existante.
- PATCH /uploads/{song_id}
Modifie la visibilité (publique ou privée) d'une musique uploadée par l'utilisateur.
- GET /uploads/{song_id}
Retourne les informations d'upload d'une musique (en respectant les règles de

visibilité).

- GET /uploads/me
Liste les musiques uploadées par l'utilisateur connecté.

7.5 Analyse

- GET /analyze/{song_id}
Retourne l'analyse prédictive stockée pour une musique donnée.
- GET /analyze/me
Retourne les musiques uploadées par l'utilisateur connecté avec leur analyse associée.
- GET /analyze/{song_id}/preview
Recalcule le score de popularité d'une musique à la volée, sans modifier la base de données.
Cette route est accessible sans connexion pour les musiques publiques.

7.6 Historique

- GET /history
Retourne l'historique de consultation des musiques de l'utilisateur connecté.
- GET /history/{song_id}
Retourne l'entrée d'historique associée à une musique précise.
- POST /history
Ajoute ou met à jour une entrée dans l'historique utilisateur.
- DELETE /history/{song_id}
Supprime une musique de l'historique.
- DELETE /history
Supprime tout l'historique de l'utilisateur connecté.

8. Machine Learning

8.1 Principe

Le modèle de Machine Learning calcule un score de popularité cohérent à partir des caractéristiques audio d'un morceau, sans chercher à produire un score artificiellement optimal.

8.2 Features utilisées

- Durée
- Tempo
- Loudness
- Key
- Mode
- Time signature

8.3 Entraînement

- Entraînement hors ligne à partir d'un dataset CSV
- Séparation train / validation / test
- Sauvegarde du modèle et de l'ordre des features

9. Frontend - Architecture et fonctionnement

9.1 Architecture

Cette application est un modèle MVC.

- **models** : ce dossier contient tous les modèles utilisés par l'application. Les modèles ne correspondent pas nécessairement à ceux utilisés par l'api, mais plutôt au contenu des réponses qu'elle renvoie. Chaque modèle possède ainsi une fonction pour formater les données de l'objet en json, et une autre permettant de transformer des données json en une instance.
- **services** : ce dossier contient tous les services nécessaires pour faire des opérations sur l'api. Les services se chargent de traiter les données fournies si besoin, les envoyer avec les bon paramètres à l'api via une requête, et traiter la réponse.
- **controllers** : ce dossier contient tous les controllers utilisés dans l'application. Ceux-ci permettent de gérer les données qui seront affichées par les vues, et de faire les appels aux services nécessaires en fonction des entrées utilisateurs.
- **views** : ce dossier contient les pages (interface) de l'application, les plus complexes (celles qui doivent gérer des données) possèdent un lien vers un controller.

Cette application possède d'autres parties complémentaires mais essentielles à son fonctionnement.

- **enums** : les énumérations qui seront utilisées dans l'application.
- **exceptions** : les exceptions personnalisées permettant de simplifier la gestion d'erreurs.
- **utils** : des fonctions utiles (comme pour la conversion de dates ou de string).
- **validators** : des fonctions permettant la validation de données, nécessaires pour les champs de formulaire.
- **widgets** : composantes d'interface spécifique à flutter, permettant une meilleur factorisation du code.

9.2 Fonction et méthodes globales

Cette application possède plusieurs fonctions et méthodes qui sont globalement utilisées dans l'application, afin de généraliser le fonctionnement, simplifier la correction d'erreurs et faciliter le développement.

- **Auth** : Permet la gestion des données de l'utilisateur actuellement connecté, et les stocker localement.
- **CustomNavigator** : définit la gestion du changement de page, afin d'avoir des animations fluides et contrôlables.
- **Provider** : permet de faire des appels à l'API de manière centralisée avec une gestion automatique du token de connexion.
- **AlertDialogBuilder** : permet d'afficher très simplement l'affichage et la fermeture de dialog (popup) depuis n'importe où dans l'application.

10. Sécurité et gestion des accès

- Authentification par cookie JWT
- Authentification optionnelle pour certaines routes
- Respect des musiques privées
- Mise à jour conditionnelle de l'historique

11. Organisation du projet

- Travail collaboratif via GitHub

- Répartition des tâches (API, ML, base de données, tests)
- Suivi via outil de gestion de projet (Trello / Notion)

Nom de la tâche	État	Personne assignée	Date d'échéance	Type de tâche	Branche d'application
Choix du thème et des données	Terminé	Marjorie MADINECOUTY	11/18/2025	Cadrage et Initialisat...	
Configuration technique	Terminé	Marjorie MADINECOUTY	11/18/2025	Cadrage et Initialisat...	
Création de schéma de base de données	Terminé	Marie Giron	11/24/2025	Conception	Back-Données
Création du diagramme de classe	Terminé	Marjorie MADINECOUTY	11/24/2025	Conception	Back-Données
Création de la base du projet BACK (python + flask)	Terminé	Alexis Chabanne	11/24/2025	Conception	Back-API
Création de la base du projet FRONT (Flutter)	Terminé	Maël	11/24/2025	Conception	Front
Création de la base de données du dataset	Terminé	Nahel	11/24/2025	Fonctionnalité	Back-Données
Design de l'interface	Terminé	Cyprien Bonnesfond	12/15/2025	Conception	Front
Création du fichier Docker	Terminé	Marie Giron	12/30/2025	Fonctionnalité	
Implémenter le modèle dans le front	Terminé	Maël	12/24/2025	Fonctionnalité	Front
Compléter le contrôleur pour la page de recherche	Terminé	Nahel	01/05/2026	Fonctionnalité	Front
Compléter la page de recherche, créer des widget si	Terminé	Nahel	01/05/2026	Fonctionnalité	Front
Créer le service pour la recherche	Terminé	Nahel	01/05/2026	Fonctionnalité	Front
Photos pour l'interface	Terminé	Maël, Alexis Chabanne	01/05/2026	Perfectionnement	Front

Lien notion :

<https://mahogany-capri-e4b.notion.site/2af964c9075e80e49776fa0639cc3b69?v=2af964c9075e80ac99fd000ce1f536a8&pvs=74>

12. Limites et perspectives

12.1 Limites

- Modèle ML volontairement simple
- Dataset de taille limitée

12.2 Perspectives

- Visualisation des données (statistiques, graphiques)
- Analyse plus poussée des corrélations
- Amélioration du modèle prédictif